Enhancement in Functionality of Conway's Game of Life

Hyoyeon Lee, Jimin Ryu
Department of Computer Science
University of Bristol
{fw22912, ff22216}@bristol.ac.uk

1. Introduction

This report provides a thorough exploration of the Game of Life (GoL) development process, utilising both parallel and distributed implementation methodologies. It delves into the details of implementing parallel and distributed designs, highlighting their functionalities and limitations. Through benchmarking results and analysis, the report offers a comprehensive understanding of how the primary objective, the enhancement of runtime performance through the effective use of worker threads, is successfully achieved.

2. Parallel Implementation

2.1 Serial Implementation and Its Limitations

The initial implementation of Game of Life followed a sequential approach, functioning as a serial implementation where each thread had to wait for the completion of the preceding thread's computation. This design offered advantages in terms of simplicity and safety by avoiding concurrent access to shared memory. However, this sequential execution resulted in an extended runtime, significantly impeding the efficiency of the project and serving as a notable limitation in the initial implementation.

2.2 Parallel Design and Functionality

A parallel implementation was built upon the serial design to improve the concurrency of the program.

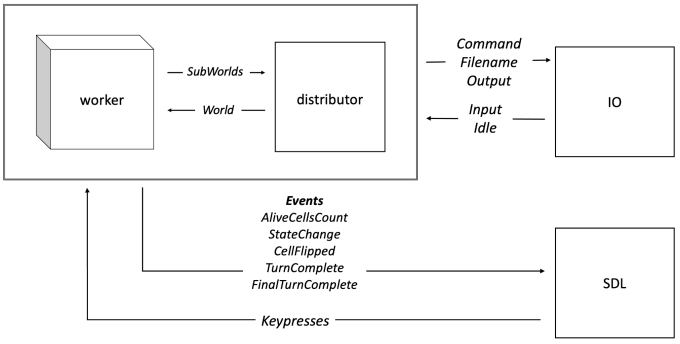

Figure 1. Implementation of parallel design

Figure 1 illustrates the overall implementation of the parallel design. The work is distributed by the distributor and is computed with multiple worker threads. The events, keypresses, and ticker are updated and notified by passing values from the distributor to the SDL via channels.

The *world* matrix was computed by allocating the starting and ending Y coordinates, assigning each worker thread the responsibility of calculating its designated portions. The resulting slices were appended together to construct a comprehensive updated world. The safe access to the world state was guaranteed via using a mutex, restricting access to the world during the updating process, and thereby avoiding potential conflicts with concurrent goroutines.

To provide periodic updates of the computed state of the world, a ticker mechanism was introduced to report the number of alive cells of the latest state every two seconds. Furthermore, another goroutine *keyPress* was also designed to detect keypress (specifically 's', 'p', and 'q') during the iteration and trigger corresponding actions.

2.2.1. Goroutine Implementation

The goroutines were strategically utilised to introduce parallelism to the program. They are used for asynchronous executions of multiple functions concurrently, including ticker, keypress, and the computation of the next world states.

Firstly, a ticker goroutine is responsible for periodically sending events of the number of alive cells and turns of the latest state of the world. Since the ticker had to be called every two seconds, it had to be adopted in a way of running both the state computation process and the ticker concurrently. By implementing this as a goroutine, this asynchronous mechanism ensures that the main execution loop, responsible for processing all turns, remains unblocked by the ticker and can proceed concurrently.

Another goroutine is employed to handle keypress events asynchronously. Through this goroutine, the program can seamlessly respond to keypress events. This asynchronous handling ensures that the program detects keypresses and facilitates a smooth user interaction without interruptions without disrupting the main computation logic.

Lastly, the computation for the next state of the world is distributed into multiple worker goroutines, each taking responsibility for calculating the next state of a designated part of the world. These goroutines enable concurrent calculation with having memory access to the same *world* matrix. This enhances calculation speed, demonstrating a notable contrast in terms of memory access and runtime compared to a serial implementation, where each worker must wait for the previous one to finish its computation before starting the next.

Overall, the use of the goroutines in this design contributes to a concurrent execution model, which improves the efficiency and decreases the runtime of the program.

2.2.2. Memory Sharing and Synchronisation

The program was designed with consideration for concurrent access to shared data structure via using mutex and channels. In the design, the role of a channel is critical to facilitate communication among goroutines.

The ticker goroutine, responsible for reporting the number of alive cells, gets triggered every two seconds via channel *t* and transmits the number of alive cells and the latest completed turn through a designated channel. Simultaneously, the keyPress goroutine manages keypress commands, such as pausing and printing out the current state by receiving the keypress value via channel.

The core of the concurrent computation via channel lies in the worker goroutines. Each worker goroutine is assigned a specific part of the world matrix to independently calculate the next state. The calculated slices are stored as a channel in an array, which allows the worker goroutines to efficiently deliver their calculated slices back to the distributor. The utilisation of channels as a communication mechanism fosters a streamlined and synchronised flow of information, which is a key factor in the efficient execution of concurrent tasks within the implementation.

2.3. Challenges During Implementation

2.3.1. Race Conditions

One issue faced during implementation was multiple goroutines having access to the same *world* matrix throughout the programme, between *countingCells* and the worker goroutines.

This issue was solved by adopting a global mutex over every function that requires access to the world. Having mutex prevented *countingCells* from accessing the uncompleted world while worker threads are still working on it. The mutex also prevented the ticker from accessing the world while the worker goroutines hadn't finished their

computation. By enforcing exclusive access by a mutex, it is ensured that the ticker would refrain from accessing the 'world' matrix before the worker goroutines completed their computations and appended the final state for each turn. This strategic application of the mutex safeguards against race conditions, guaranteeing the consistency of shared memory access throughout the program execution.

2.3.2 Work Distribution

In the initial implementation of the work distribution, a notable deficiency arose concerning the equitable distribution of workload among workers. In situations where the workload couldn't be evenly distributed, the design assigned the same tasks to all workers except the last one. The final worker was then responsible for handling all the remaining workload. It was observed that this approach introduced inefficiencies, resulting in an increase in overall runtime.

To address this issue, a refined implementation was conceived, where threads of an amount of the remainder were obtained by dividing the height of the input by the number of threads being assigned an extra row of work equally. The primary objective was to solve uneven workload distribution, ensuring that no single thread undertook a disproportionately large share of the remaining tasks.

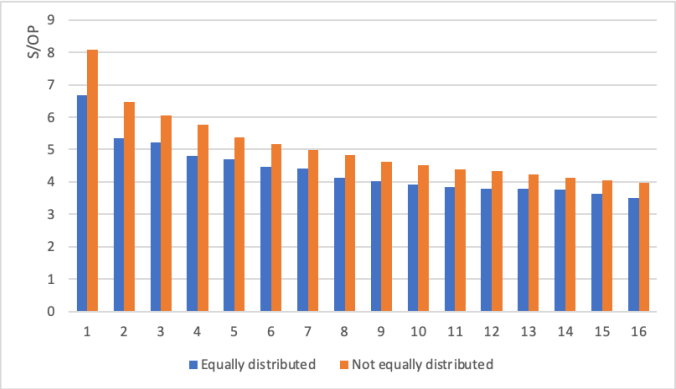

Figure 2. Variation of s/op with number of threads

The results are rounded up to four decimal places. The following benchmark test was repeated 10 times on a 12-core i7-1260P machine. This test was designed to proceed with a 512x512 PGM file from one thread up to 16 threads.

According to Figure 2, both the initial and latest implementations exhibit a downward-sloping curve, reaching a peak in the case of a single worker thread. The average s/op stood at 5.0625 seconds for not equally distributed design, whereas the equally distributed implementation achieved an average of 4.3783 seconds per operation, resulting in an average of 0.6842 lower runtime.

The consequential outcome of this implementation adjustment was marked as an improvement in the distribution process of workload among worker goroutines, fostering a more balanced allocation of tasks. This not only mitigated the inefficiencies in terms of runtime associated with the previous uneven distribution but also fostered an enhanced parallelism. The result of these adjustments is reflected in the reduction in the overall runtime of the parallelised computation.

2.4. Variation of CPU Time With Number of Threads

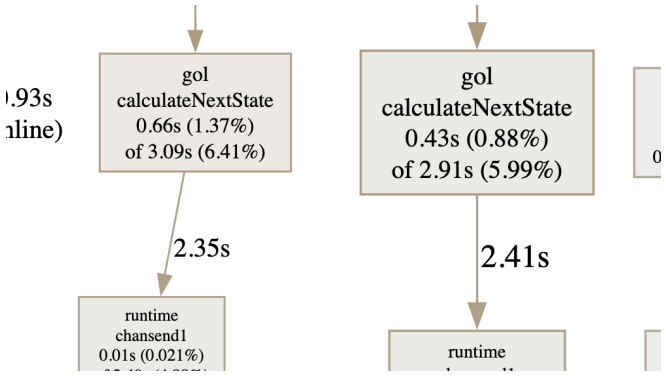

Figure 3. Comparison of CPU profile between 8-workers(left) and 16-workers(right)

The diagram shows the percentage of CPU time consumed, where the 16-worker thread implementation showed 0.23s shorter processing time compared to 8 worker thread implementation. By this result, it was assumed that doubling the number of threads does not necessarily halves the runtime, whereas not the whole program but only a

portion of it is parallelised, resulting in a limitation of the speedup achieved via parallel design.

2.5. Conclusion

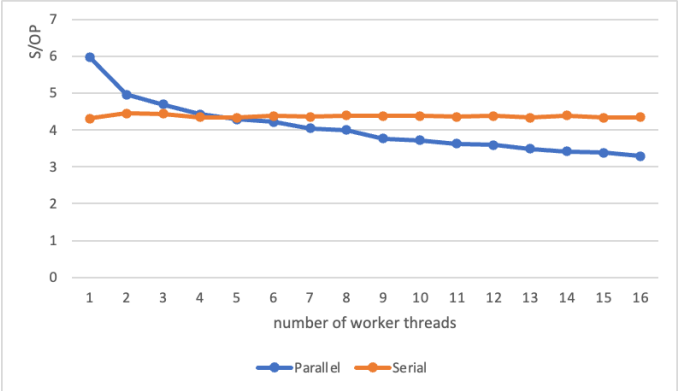

Figure 4. Comparison between serial and parallel implementations for 1000 turns

The results of the graph are bounded up to 4 decimal places.

The runtime of parallel implementation has shown a downward sloping curve as it reached its peak at 1-worker thread with 5.9785 s/op, while the 16-thread iteration had the lowest runtime of 3.2936 s/op. Serial implementation, on the other hand, showed a nearly linear relationship, with the highest and lowest value showing the difference of 0.1146 s/op.

The purpose of this experiment was to see the relationship between the runtime and the number of threads and make a comparison between serial and parallel implementation.

The assumption that the average runtime of the parallel implementation would be smaller than the serial implementation when there are multiple worker threads, was supported through this simulation.

3. Distributed Implementation

3.1. Functionality and Design

The implementation of the distributed design started with segregating the serial implementation into the corresponding parts, either being in a server or distributor. This transition involved deploying the server-client system to facilitate communication and coordination between the local controller and the GOL Engine. Communication among system components is accomplished through RPC(Remote Procedure Call) connections, which are facilitated by the stubs.go file that defines common message types and interfaces.

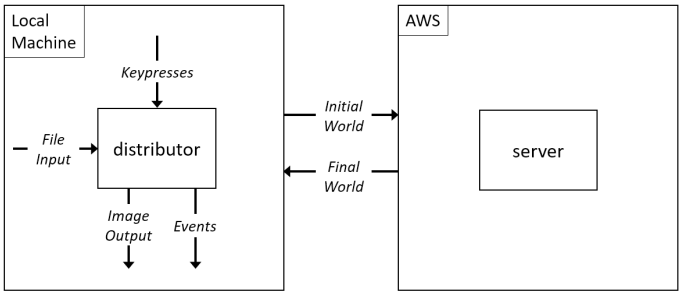

Figure 5. Diagram of single-threaded, single-machine implementation

The initial implementation operated in a single-threaded, single-machine setup. The local controller sends the initial world state to the GOL Engine, which then calculates the next state and returns the result to the local controller.

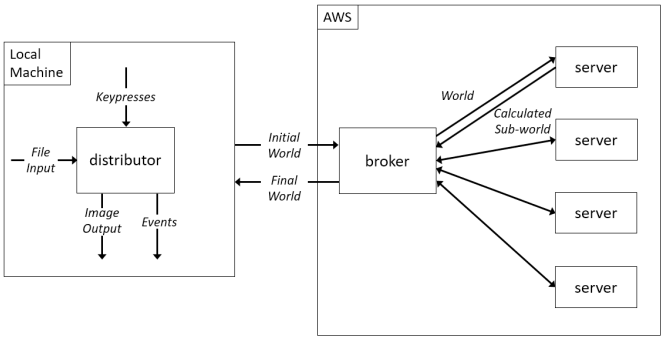

Figure 6. Diagram of multiple worker machines with broker implementation

To enhance system scalability, we implemented a configuration that involves four servers that would handle the overall computation of the *world*.

Additionally, a broker was introduced, located between the servers and a distributor playing a crucial role in facilitating communication between servers and distributors. The main responsibility of the broker was being responsible for distributing tasks such as assigning specific world ranges to the server and reassembling slices of the world, which are returned to the distributor.

The process starts with the distributor sending the initial state of the world to the broker. Following this, the broker assigns particular world ranges to each server by transmitting specified Y coordinates with the world. Each server then computes the next state of the world and transmits the computed slice back to the broker. The broker returns the updated world to the distributor, completing the cycle of distributed computation.

Unlike parallel design, the channel implementation was not suitable as a communication method for the distributed program, where a new method was needed. Therefore, the communication method for the design was replaced from channel-based communication with RPC connections. This modification enabled effective communication between multiple workers, the broker, and the distributor.

3.2 Challenges During Implementation

3.2.1 Establishing Multiple Worker Connections

Since there are multiple servers, it became crucial for the broker to establish the correct connection with all the servers. This issue was solved by declaring a slice of server pointers, where an anonymous function connecToServer adds a successfully connected server to the slice. Retrieving the current server and incrementing the index of the slice allowed it to effectively address the current server correctly, preventing misconnection or producing wrong RPC requests.

3.2.2 Implementing Keypress Events

During the implementation of the keypress event 'k', there was a challenge with the graceful shutdown of the distributed systems. To achieve this, it was crucial to sequentially shut down the server, broker, and distributor. It was observed that an improper sequence of shutdown could lead to incomplete termination.

The initial implementation had the broker being shut down first, resulting in the failure of the server to terminate despite the successful shutdowns of the broker and distributor. Conversely, when the servers were shut down initially, the broker had difficulties terminating properly, resulting in an EOF error with the broker.

A separate RPC was adopted to address this issue, first starting with the distributor creating an RPC after receiving the keypress value ‘k’. This triggers another RPC from the broker to the servers, with *ExitServerHandler* that gracefully shutdown the servers. Both servers and a broker have a local channel, which alerts a goroutine that the program is ready to terminate. This effectively initiated a systematic and sequential shutdown of the program, which is a desired reaction for key value ‘k’.

3.3. Benchmark Results and Analysis

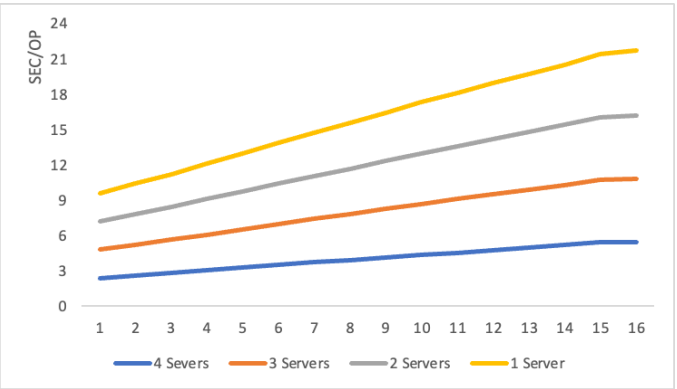

Figure 7. Stacked line graph; Variation of s/op with the number of threads with multiple servers

The benchmark test was done on the Intel I7-8700 university lab machine.

According to Figure 7, all four cases showed an upward-sloping curve as the number of threads grew. Since the current implementation does not support parallel computation among the servers, the current design is more close to serial computation. However, as the slope becomes gentler as more servers are added, it is indicated that the design has a degree of scalability and results in improved performance as the system scales.

3.4. Fault Tolerance

Fault tolerance is implemented for the 'q' keypress event to ensure continued operation even if the controller is terminated. Upon closing the controller client program, a new controller seamlessly takes over interaction with the GoL engine, and calculations continue from the previous world state.

To facilitate fault tolerance, new global variables *resumeWorld* and *resumeTurn* are declared in the broker to store the latest state of the world and the turn. In the distributor, an RPC call is initiated to the broker, which checks whether *resumeWorld* is empty. If it is empty, the distributor proceeds with the original RPC call using the initial state of the world and turns, indicating that the broker has not run before. Conversely, if *resumeWorld* is not empty, implying that the broker has run before and holds the latest state of the world and turn, the broker returns these values in its response, which are then transmitted to the distributor. The distributor subsequently makes an RPC call with these values, facilitating continuous iteration from the previous state and ensuring fault-tolerant behaviour.

3.5. Limitations and Potential Improvements

The decision to implement four servers in the distributed design was motivated by the potential scalability offered by multiple worker threads. However, despite this advantage, the current distributed design still falls short in terms of concurrency and parallelism, resembling the characteristics of a serial implementation. This is evident as only one RPC is issued and processed sequentially as it is reflected in Figure 7. To overcome this limitation, a potential solution involves adding parallel workers within each distributed AWS Node, allowing for the concurrent execution of multiple workers.

3.6. Conclusion

In our exploration of distributed implementation, we have identified several advantages, such as heightened fault tolerance, redundancy, and scalability. These qualities collectively contribute to achieving high availability, effective load balancing, and the ability to accommodate heightened demand, ultimately enhancing the overall performance of the system. While there are opportunities for improvement, the findings through this simulation validate the positive advancements made in reinforcing the system's overall functionality.